
Python 金融数据分析

第 3 章 NumPy 数组计算

主讲人： 王伟冠, 吕文俊
院 系： 经济学院金融系

上海大学

2026 年 9 月 24 日



1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
7. 综合案例与小结



1. 为什么需要 NumPy?

2. 创建数组

3. 索引与切片

4. 向量化运算

5. 布尔索引：按条件筛选

6. 统计函数

7. 综合案例与小结



- 理解为什么金融数据分析离不开 NumPy
- 掌握 ndarray 的创建、属性与索引切片
- 掌握**向量化运算**：一行代码对整个数组做计算
- 掌握**布尔索引**：按条件筛选数据
- 掌握常用统计函数（均值、标准差、累计等）
- 能够用 NumPy 计算收益率序列与波动率等风险指标

前置知识

第 2 章的列表、循环、函数——本章会把它们“升级”为数组写法。

```
1 prices = [100.0, 102.0, 101.0, 105.0]
2 returns = []
3 for i in range(1, len(prices)):
4     r = (prices[i] - prices[i-1]) / prices[i-1]
5     returns.append(r)
```

- 只有 4 个价格没问题；但真实数据动辄数千个交易日、成百上千只股票。
- Python 的 for 循环每一步都要检查类型、逐个搬运，**速度慢**；
- 代码冗长，意图（“算收益率”）被循环细节淹没。

本章的目标

用 NumPy 把 4 行循环压缩成 1 行，并且快上百倍。

- **NumPy** (Numerical Python) 是 Python 科学计算的基石。
- 核心对象: ndarray——存储**同一类型**数据的多维数组。
- 底层用 C 实现, 运算直接作用于整块内存, 速度远超 Python 循环。
- pandas、scikit-learn 等后续工具都构建在 NumPy 之上。

```
1 import numpy as np      # 约定俗成的导入方式
2
3 prices = np.array([100.0, 102.0, 101.0, 105.0])
4 returns = prices[1:] / prices[:-1] - 1    # 一行搞定!
```

约定

`import numpy as np` 是全世界通用的写法, 请照抄。

1. 为什么需要 NumPy?
- 2. 创建数组**
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
7. 综合案例与小结



```
1 a = np.array([100.0, 102.0, 101.0, 105.0])
2 a          # array([100., 102., 101., 105.])
3
4 type(a)     # numpy.ndarray
5 a.shape     # (4,)      形状: 长度为4的一维数组
6 a.dtype     # float64   元素类型: 全数组统一
7 len(a)      # 4
```

- `np.array(列表)`: 把 Python 列表转换为数组。
- **类型统一**: 数组中所有元素同一类型 (如全是 `float64`), 这正是快的原因。
- `shape` 描述"形状", 是后面理解二维数据的关键。


```
1 np.zeros(5)           # array([0., 0., 0., 0., 0.])
2 np.ones(3)            # array([1., 1., 1.])
3
4 np.arange(0, 10, 2)    # array([0, 2, 4, 6, 8]) 类似range
5 np.linspace(0, 1, 5)   # array([0., 0.25, 0.5, 0.75, 1.])
6                        # 0到1之间均匀取5个点（含端点）
```

- zeros/ones: 初始化占位数组（如预存结果）。
- arange(起, 止, 步长): 整数序列, 与 range 类似但返回数组。
- linspace(起, 止, 个数): 画利率区间、画函数图像时常用。

金融场景

给一组折现率 `np.linspace(0.01, 0.10, 10)`, 批量观察债券价格变化。

```
1 # 3天 x 2只股票 的收盘价：每行一天，每列一只股票
2 p = np.array([[25.0, 100.0],
3               [25.5, 99.0],
4               [26.0, 102.0]])
5
6 p.shape      # (3, 2)    3行2列
7 p.ndim       # 2        维度数
8 p.size       # 6        元素总数
```

- 列表中套列表 → 二维数组，可以看作**表格/矩阵**。
- `shape` 变为（行数，列数）。
- 金融数据的典型形态：**行 = 日期，列 = 资产**。

每行是一名学生，每列是一门课。axis=0 沿学生方向汇总，得到各科均分；axis=1 沿科目方向汇总，得到各学生均分。

```
1 scores = np.array([[80, 90, 70], [60, 75, 90]])
2 print(scores.shape)           # 2名学生、3门课
3 print(scores.mean(axis=0))    # [70. 82.5 80.]
4 print(scores.mean(axis=1))    # [80. 75.]
```

```
1 a = np.arange(6)           # [0 1 2 3 4 5]
2 a.reshape(2, 3)           # 变成2行3列
3 a.reshape(-1, 2)          # -1 表示"自动算": 3行2列
4
5 np.array([1, 2]) + 0       # 数组拼接用专门函数:
6 np.concatenate([np.array([1, 2]), np.array([3, 4])])
7                           # array([1, 2, 3, 4])
```

易错点

数组的 + 是**逐元素相加**，不是拼接！拼接请用 `np.concatenate`。
`np.array([1,2]) + np.array([3,4])` 得到 `[4, 6]`。

1. 为什么需要 NumPy?
2. 创建数组
- 3. 索引与切片**
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
7. 综合案例与小结



```
1 prices = np.array([30.1, 30.5, 29.8, 31.2, 30.9])
2
3 prices[0]          # 30.1    索引从0开始
4 prices[-1]         # 30.9    倒数第一个
5 prices[1:3]        # array([30.5, 29.8])  含头不含尾
6 prices[1:]         # 除第一个外的所有元素
7 prices[:-1]        # 除最后一个外的所有元素
```

- 索引、切片规则与列表完全一致——旧知识直接复用。
- `prices[1:]` 与 `prices[:-1]` 这对组合后面算收益率要用，务必熟练。

注意

数组切片返回的是**视图**（与原数组共享内存）：修改切片会影响原数组，与列表不同。

```
1 p = np.array([[25.0, 100.0],
2               [25.5, 99.0],
3               [26.0, 102.0]])
4
5 p[0, 0]      # 25.0   第0行第0列
6 p[1, 1]      # 99.0   第1行第1列
7 p[0]         # 第0行 -> array([25., 100.])
8 p[:, 1]      # 所有行、第1列 -> 股票2的价格序列
9 p[1:, :]     # 第1行到末尾的所有行
```

- 写法：数组 [行, 列]；冒号：表示“全部”。
- `p[:, 1]` 即“取出某只股票的整个价格序列”，金融分析中的高频操作。

基本切片通常共享原数组的数据。需要独立修改时加 `copy()`。这与 Python 列表的切片不同。

```
1 measurements = np.array([10, 20, 30, 40])
2 view = measurements[1:3]
3 independent = measurements[1:3].copy()
4 view[0] = 99
5 print(measurements) # [10 99 30 40]
6 print(independent)  # [20 30]
```


改成 `measurements[[1, 2]]` 后再修改结果，原数组会变化吗？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
- 4. 向量化运算**
5. 布尔索引：按条件筛选
6. 统计函数
7. 综合案例与小结



```
1 prices = np.array([100.0, 102.0, 101.0, 105.0])
2
3 prices + 1      # 每个元素加1
4 prices * 1.05   # 每个元素涨5%
5 prices ** 2     # 每个元素平方
6 1 / prices      # 每个元素取倒数
```

- **向量化**：运算自动作用于数组中的**每一个**元素，无需循环。
- 运算不改变原数组，而是返回**新数组**。
- 读代码时把数组想象成“一整列 Excel 数据同时参与计算”。

```
1 a = np.array([1.0, 2.0, 3.0])
2 b = np.array([10., 20., 30.])
3
4 a + b          # array([11., 22., 33.])
5 a * b          # array([10., 40., 90.])
6 b / a          # array([10., 10., 10.] )
```

- 两个数组运算：**位置一一对应**（第 0 个对第 0 个、第 1 个对第 1 个……）。
- 这正对应金融场景：
 - “今日价 / 昨日价”：两个错开一位的价格序列相除；
 - “组合收益”：持仓权重数组 $\times$ 个股收益数组。

形状要匹配

参与运算的两个数组形状必须一致（或满足广播规则），否则报错。

(2, 3) 与 (3,) 可以逐列配对。广播从末尾维度比较，长度相同或其中一个为 1 才能兼容。

```
1 scores = np.array([[80, 90, 70], [60, 75, 90]])
2 bonus = np.array([2, 0, 5])
3 adjusted = scores + bonus
4 print(adjusted)
5 student_mean = scores.mean(axis=1, keepdims=True)
6 print(student_mean.shape) # (2, 1)
7 print(scores - student_mean)
```

将每一科减去该科均分；解释结果为什么每列均值为 0。

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

```
1 prices = np.array([100.0, 102.0, 101.0, 105.0])
2
3 # 简单收益率：(今日-昨日)/昨日
4 ret = prices[1:] / prices[:-1] - 1
5 ret      # array([0.02, -0.0098, 0.0396])
6
7 # 对数收益率（后文常用）
8 logret = np.log(prices[1:] / prices[:-1])
```

- `prices[1:]`：今日价序列；`prices[:-1]`：昨日价序列；相除即逐日配对。
- 上一章的 4 行循环，现在 1 行；数据越大，优势越明显。
- `np.log()` 是向量化函数，对整个数组取自然对数。

```
1 big = np.arange(1_000_000)           # 0到999999
2
3 %timeit [x ** 2 for x in big]         # Python循环: 约200毫秒
4 %timeit big ** 2                     # NumPy向量化: 约1毫秒
```

- `%timeit` 是 Notebook 计时魔法命令（下节 notebook 中可亲自体验）。
- 同样的一百万元素平方运算，NumPy 快约 **100 倍以上**。
- 原因：类型统一 + 底层 C 实现 + 无需逐步类型检查。

经验法则

看到“对每个元素做同样的事”，先想向量化，再考虑循环。

函数	作用	金融用途举例
<code>np.log(x) / np.exp(x)</code>	对数 / 指数	对数收益率、连续复利
<code>np.sqrt(x)</code>	平方根	波动率换算
<code>np.abs(x)</code>	绝对值	偏差幅度
<code>np.maximum(a,b)</code>	逐元素取大	期权收益 $\max(S - K, 0)$
<code>np.round(x, n)</code>	四舍五入	报表展示
<code>np.cumsum(x)</code>	累计和	累计收益、净值曲线
<code>np.diff(x)</code>	相邻差分	价格变动额

期权到期收益

`np.maximum(S - K, 0)`: 行权价 K , 到期价序列 S , 一行算出所有情形的收益。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
- 5. 布尔索引：按条件筛选**
6. 统计函数
7. 综合案例与小结



```
1 ret = np.array([0.02, -0.01, 0.04, -0.03, 0.01])
2
3 ret > 0      # array([True, False, True, False, True])
```

- 数组的比较运算也是**向量化**的：逐元素判断，返回等长的布尔数组。
- 布尔数组可以当“筛选器”使用（下一页）。
- 组合条件要用 &（且）、|（或），且每个条件加括号：

```
1 (ret > -0.01) & (ret < 0.01)      # 小幅波动日
```

易错点

数组条件中 and、or 会报错，必须用 &、|。

```
1 ret = np.array([0.02, -0.01, 0.04, -0.03, 0.01])
2
3 ret[ret > 0]          # array([0.02, 0.04, 0.01]) 只留上涨日
4 ret[ret < -0.02]      # array([-0.03])          大跌日
5
6 np.sum(ret > 0)        # 3      True按1计数: 上涨天数
7 (ret > 0).mean()       # 0.6    上涨日占比
```

- 数组 [布尔数组]: 只保留对应位置为 True 的元素。
- `np.sum(布尔数组)`: 统计满足条件的个数——非常实用的技巧。
- 金融应用: 统计涨停天数、回撤超过阈值的次数、极端波动日。

```
1 ret = np.array([0.02, -0.01, 0.04])
2
3 np.where(ret > 0, 1, 0)
4 # array([1, 0, 1]) 条件为真取1, 否则取0
5
6 np.where(ret > 0, "涨", "跌")
7 # array(['涨', '跌', '涨'])
```

- np.where(条件, 真时的值, 假时的值): 向量化的 if/else。
- 应用: 构造交易信号——收益为正记 1 (持有多头), 为负记 0。
- 这比写 for 循环套 if 简洁且快得多。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
- 6. 统计函数**
7. 综合案例与小结



```
1 ret = np.array([0.02, -0.01, 0.04, -0.03, 0.01])
2
3 ret.sum()          # 0.03      求和
4 ret.mean()         # 0.006     均值（日均收益率）
5 ret.std()          # 0.026... 标准差（波动性）
6 ret.max()          # 0.04      最大值
7 ret.min()          # -0.03     最小值
8 ret.argmax()       # 2         最大值的“位置”
```

- “把一串数压缩成一个数”的函数叫**聚合函数**。
- 写法两种等价：`ret.mean()` 或 `np.mean(ret)`。
- **标准差**是金融中风险（波动率）的度量，务必熟悉。

```
1 # 3天 x 2只股票的日收益率
2 r = np.array([[ 0.02, -0.01],
3               [-0.01,  0.03],
4               [ 0.04,  0.01]])
5
6 r.mean()          # 0.0133... 全部元素的均值
7 r.mean(axis=0)    # 按"列"求均值 -> 每只股票的日均收益
8 r.mean(axis=1)    # 按"行"求均值 -> 每天的组合平均收益
```

- axis=0: 沿行的方向压缩 → 结果对应每一列;
- axis=1: 沿列的方向压缩 → 结果对应每一行。
- 记忆口诀: axis 是被"压掉"的那一维。


```
1 prices = np.array([100., 102., 101., 105.])
2
3 np.diff(prices)      # array([2., -1., 4.]) 每日涨跌"额"
4 np.cumsum([1, 1, 1, 1]) # array([1, 2, 3, 4]) 累计和
```

- `np.diff`: 相邻元素相减, 长度少 1——快速得到每日变动。
- `np.cumsum`: 逐步累加——从"每日现金流"得到"累计现金流", 从"对数收益率"恢复"累计对数收益"。

小例子

对数收益率序列取 `np.cumsum`, 再取 `np.exp`, 即得净值曲线 (相对期初价格)。

每步向左或向右移动一格，起点为 0。先生成步长，再用 `cumsum` 得到位置；随机种子使课堂输出可复现。

```
1 walk_rng = np.random.default_rng(7)
2 directions = walk_rng.integers(0, 2, size=12)
3 steps = np.where(directions == 0, -1, 1)
4 positions = np.r_[0, steps.cumsum()]
5 print(steps)
6 print(positions)
7 print("最远距离: ", np.abs(positions).max())
```

改成 100 步，并计算走动后位于起点右侧的比例；分母是否包括起点？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

1. 为什么需要 NumPy?
2. 创建数组
3. 索引与切片
4. 向量化运算
5. 布尔索引：按条件筛选
6. 统计函数
- 7. 综合案例与小结**



```
1 rng = np.random.default_rng(8)          # 随机数生成器(固定种子)
2 n = 250                                  # 250个交易日
3
4 # 模拟日收益率：均值0.04%，日波动1.5%
5 rets = 0.0004 + 0.015 * rng.standard_normal(n)
6
7 # 由收益率还原价格曲线（期初100元）
8 prices = 100 * np.exp(np.cumsum(rets))
9
10 print(f"日均收益率：{rets.mean():.4%}")
11 print(f"日波动率： {rets.std():.4%}")
12 print(f"年化波动率：{rets.std()*np.sqrt(252):.2%}")
13 print(f"最高价/最低价：{prices.max():.2f} / {prices.min():.2f}")
```

- **年化波动率** = **日标准差** $\times \sqrt{252}$ ，业界标准做法。
- 用到的全部是本章工具：向量化、cumsum、exp、聚合函数。

```
1 # 上涨日占比
2 print(f"上涨日占比: {(rets > 0).mean():.2%}")
3
4 # 单日跌幅超过3%的"极端日"
5 extreme = rets[rets < -0.03]
6 print(f"极端下跌天数: {len(extreme)}")
7
8 # 最大回撤的粗略观察: 历史最高价 vs 当前价
9 peak = np.maximum.accumulate(prices)
10 drawdown = prices / peak - 1
11 print(f"最大回撤: {drawdown.min():.2%}")
```

- `np.maximum.accumulate`: 到每个时点为止的历史最高价。
- **最大回撤**是基金与策略最常用的风险指标之一。
- 整段分析没有写一个 `for` 循环。

- **创建**: `np.array`、`zeros/ones`、`arange/linspace`; 属性 `shape/dtype`。
- **索引切片**: 与列表同规则; 二维写 [行, 列], `:` 表示全部。
- **向量化**: 数组运算逐元素进行, 一行顶循环, 速度快百倍;
数组的 `+` 是相加不是拼接。
- **布尔索引**: `a[a>0]` 筛选; `np.sum(条件)` 计数; 条件组合用 `&`、`|`。
- **统计**: `mean/std/max/min`; `axis` 是被压掉的维度。
- **金融工具箱**: 收益率 `p[1:]/p[:-1]-1`、年化波动率 $\sigma \times \sqrt{252}$ 、`cumsum/diff`、`np.where` 信号。



课后任务

- ① 运行课堂 notebook，修改参数观察输出变化。
- ② 给定价格数组 `np.array([50, 52, 51, 55, 53, 56])`：
计算日收益率、找出最大单日涨幅、统计上涨天数。
- ③ 选做：用 `np.where` 构造信号“收益为正买 1 手、为负卖 1 手”，
并计算该策略的每日收益（信号 $\times$ 次日收益，注意错位）。

下章预告

数组只装数字，真实行情还有日期、股票代码、公司名。
下一章学习 **pandas**：带标签的表格数据，金融数据分析的主力工具。

本章内容改编自教材第 4 章^[1]。

- Wes McKinney, *Python for Data Analysis*, 3rd ed.
- <https://wesmckinney.com/book/numpy-basics>
- 新增中文说明、合成数据与练习为本课程自行编写。
- 阅读正文理解概念；在配套 Notebook 中运行、修改、核对。



- [1] 伊夫·希尔皮斯科. Python 金融大数据分析 [M]. 2nd ed. 北京: 人民邮电出版社, 2018.



Thanks for your attention!

Q & A

